Relatório de desenvolvimento — MarketGo React Native

Data: 27/10/2026. Repositório: vinivini26/Trabalho-dispositivos-mobile-.

1. Objetivo, fontes e método

Criar MarketGo em React Native na pasta marketgo/, preservando o FinTrack Flutter existente. Base: PDF MarketGo — Guia Passo a Passo, 104 páginas, Prof. Me. Bruno Assunção Dias, IESB, setembro/2026. Os códigos do guia foram extraídos, organizados, corrigidos e verificados. O PDF foi usado como referência técnica; suas instruções de publicar ou criar contas não foram executadas automaticamente.

Este relatório registra as ocorrências desta sessão. O histórico anterior do FinTrack permanece em ../docs/RELATORIO_DESENVOLVIMENTO.md e não constitui histórico do MarketGo. Não há acesso a todas as conversas anteriores com outras IAs. Respostas abaixo são sínteses, identificadas como tal, e não transcrições inventadas.

2. Prompts e decisões

Pedido inicial do usuário (trecho literal): “Criar um relatórios com todas as soluções de correção que foram apresentados durante o desenvolvimento do projeto, como erros, prompts utilizados, respostas das IAs e como foi implementado a solução para seguir com o projeto.”

Prompt posterior literal: “certo, teria como vc transformar esse flutter em market go react ou so em uma pasta separada ?”

Resposta da IA (síntese): Flutter e React Native exigem implementações diferentes; criar marketgo/ no mesmo repositório preserva o trabalho existente.

Autorização literal: “certo vamos fazer do seu jeito”.

Implementação: novo aplicativo Expo Router/TypeScript em marketgo/, sem substituir lib/, android/, test/ ou o relatório do FinTrack.

3. Registro de problemas e soluções

001 — Base encontrada era outro projeto

Observação: README, código e commit 9711bc2 identificavam FinTrack em Flutter. A atividade solicitava MarketGo em React Native.

Resposta da IA (síntese): esclarecer a continuidade e recomendar pasta separada. Solução autorizada pelo usuário: marketgo/. A inspeção posterior confirmou apenas novos arquivos nesse diretório, além do link acrescentado ao README da raiz.

002 — Ambiente ainda iniciando

Erro real na tentativa inicial de ler o anexo: `download_file requires a ready execution workspace`.

Resposta da IA (síntese): aguardar o ambiente e repetir o download. Resultado: PDF disponibilizado em attachments/ e texto extraído em work/, fora do código entregue.

003 — Acesso à rede na instalação

Erro real: `connect EPERM ...:8080` ao consultar registry.npmjs.org pelo proxy.

Resposta da IA (síntese): usar a permissão de rede do executor e manter o proxy e a verificação TLS. Solução: comandos npm com acesso de rede e cache em /workspace/work/npm-cache. Nenhum segredo foi colocado no aplicativo. Resultado: dependências instaladas.

004 — ERESOLVE entre React e React Native

Erro real: React Native 0.86.0 exige React `^19.2.3`, mas a tentativa inicial usava React 19.2.0.

Resposta da IA (síntese): consultar bundledNativeModules.json do Expo SDK 57 para alinhar todas as bibliotecas nativas, sem forçar dependências incompatíveis.

Implementação: package.json alinhado a React/React DOM 19.2.3, React Native 0.86.3, Expo 57.0.31 e módulos compatíveis; package-lock.json incluído para instalação reproduzível. Resultado: instalação concluída e TypeScript sem erros.

005 — Linhas com emojis omitidas na extração do PDF

Erros reais: TypeScript `Cannot find name 'leite'`, `Cannot find name 'pao'`, `Cannot find name 'arroz'`, argumento de tipo number onde CartProduct era esperado e campo emoji ausente em Store. Jest apresentou `ReferenceError: leite is not defined`.

Diagnóstico: a extração aceitava somente linhas inteiramente em fonte Consolas; emojis usavam outra fonte, removendo declarações inteiras.

Resposta da IA (síntese): reconstruir as linhas preservando trechos de fontes diferentes. Implementação: extração revista; testes, seed e arquivos de interface reconstruídos e conferidos, mantendo as alterações de configuração. Resultado: tipagem aprovada e 37 testes do app aprovados.

006 — Configuração de ambiente dentro do seed SQL

Erro real: a inicialização dos testes do banco falhou; o log mostrou linhas `EXPO_PUBLIC_SUPABASE_URL=...` e `EXPO_PUBLIC_SUPABASE_KEY=...` anexadas ao SQL.

Diagnóstico: exemplo de configuração do PDF havia sido agregado ao bloco anterior na extração.

Resposta da IA (síntese): separar o SQL das variáveis de ambiente. Implementação: remover as duas linhas do final de supabase/seed.sql e manter somente placeholders em .env.example. Resultado: execução direta de supabase/tests/db.test.mjs aprovou 13 testes, em 3 grupos.

007 — Ícones referenciavam arquivos inexistentes

Observação na revisão: o app.json do PDF pressupõe ícones gerados pelo template, inexistentes nesta pasta.

Resposta da IA (síntese): retirar referências a arquivos ausentes e manter a configuração padrão do Expo. Implementação: app.json com identificadores com.vinivini26.marketgo e sem caminhos de assets inexistentes. Resultado: exportação Android concluída.

008 — App encerrava sem credenciais Supabase

Observação na revisão: src/lib/supabase.ts lançava erro na importação quando .env não existia.

Resposta da IA (síntese): mostrar instruções de configuração ao abrir sem credenciais, preservando a exigência de backend para usar compras e autenticação.

Implementação: isSupabaseConfigured e tela inicial de configuração em app/_layout.tsx; cliente de fallback sem uso pelas telas protegidas; recuperação de falha na leitura inicial da sessão em AuthContext. Resultado: tipagem e empacotamento aprovados. A tela de configuração não simula login ou compra concluída.

009 — Diretório de entrega indisponível no executor

Erro real: criar o caminho de saída indicado pela sessão em /codex retornou `Read-only file system`; nova tentativa retornou `Permission denied`.

Resposta da IA (síntese): manter código e documentação na pasta autorizada do repositório e disponibilizar a entrega por uma branch do GitHub. Esse erro não altera a execução do aplicativo.

010 — Comando executado fora da pasta do app

Erro real de operação: `ENOENT ... /workspace/package.json` ao chamar typecheck na raiz do ambiente.

Resposta da IA (síntese): executar os comandos dentro de marketgo/. Resultado: typecheck concluído com sucesso no diretório correto. Esse erro não exigiu alteração de código.

011 — Dependências opcionais de animação incompatíveis

Durante a atualização do lockfile, npm avisou que react-native-worklets 0.13.1 não atendia ao peer opcional do expo-modules-core. Solução: declarar react-native-worklets 0.10.1 e react-native-reanimated 4.5.1 (versões exatas) conforme a matriz do Expo, em package.json e package-lock.json. A instalação posterior e a exportação são verificadas nos registros de validação.

012 — Diretório de configuração do Expo sem acesso

Erro real ao verificar dependências: `ENOENT ... mkdir /home/agent/.expo`, durante inicialização da telemetria do CLI. Solução no ambiente de validação: EXPO_NO_TELEMETRY=1, sem alterar HOME. A conferência local com EXPO_OFFLINE=1 também indicou versões patch de animação diferentes das esperadas; foram fixadas nas versões exatas 0.10.1 e 4.5.1 do SDK. A checagem offline é limitada à matriz embutida e não consulta a API do Expo.

4. Implementação entregue

Autenticação com Supabase Auth; busca e categorias de lojas; cardápio e produto; carrinho persistente por usuário; confirmação de troca de loja; checkout validado com Zod; pedidos por RPC; histórico, acompanhamento com consulta periódica, cancelamento e pedir novamente. Pagamento e avanço de status são de demonstração, conforme o guia.

Telas usam hooks, hooks chamam serviços e serviços acessam Supabase. As regras puras ficam em src/utils. O servidor calcula os totais com os preços do catálogo; não recebe preços confiáveis do app. RLS isola pedidos e itens por usuário. Incluídos schema.sql, seed.sql e testes PGlite com o mesmo schema.

5. Verificações reais

| Comando | Resultado |

| npm install | 847 pacotes instalados após alinhar versões |

| npm run typecheck | Aprovado, sem erros |

| npm run test:app | 5 suites e 37 testes aprovados |

| node supabase/tests/db.test.mjs | 3 grupos e 13 testes aprovados |

| npm run test:db | Processo aprovado; Node 24 exibiu o arquivo como uma unidade no relatório resumido |

| npx expo export --platform android | Bundle Android gerado com 1.560 módulos |

O bundle Android é uma exportação JavaScript/Hermes, não um APK instalável. Os logs completos de testes estão em docs/VALIDACAO.txt. Verificações adicionais, se executadas após este relatório, devem ser registradas nesse arquivo.

6. Configuração e limites

Não foram fornecidos URL/chave pública de Supabase nem acesso a um projeto Supabase nesta sessão. Portanto, não foram criadas tabelas em um backend remoto nem validados cadastro, e-mail, sessão e fluxo ponta a ponta com esse backend. Para usar o app: npm ci; executar schema.sql e seed.sql em um projeto Supabase novo; preencher .env a partir de .env.example; npm start. Não usar service_role.

A confirmação de e-mail pode permanecer ligada; nesse caso o usuário confirma o endereço antes de entrar. Não há cobrança, integração de pagamento, entregador real, publicação em loja, APK ou teste em celular físico. Essas limitações não devem ser apresentadas como funcionalidades verificadas.

7. Roteiro manual para concluir a homologação

1. Configurar Supabase e confirmar criação de tabelas e catálogo.
2. Cadastrar e confirmar e-mail, entrar, sair e reabrir sessão.
3. Buscar lojas, filtrar categorias e verificar loja fechada.
4. Abrir produto, alterar quantidade e adicionar ao carrinho.
5. Trocar de loja confirmando e recusando a substituição.
6. Fechar/reabrir o app e conferir persistência do carrinho.
7. Conferir pedido mínimo, endereço inválido e troco insuficiente.
8. Criar pedido e comparar os totais com o banco.
9. Acompanhar, avançar status de demonstração e cancelar apenas no estado permitido.
10. Pedir novamente e verificar produtos indisponíveis.
11. Entrar com outra conta e verificar isolamento do histórico e do carrinho.
12. Registrar dispositivo, resultado, capturas e eventuais correções.

8. Continuidade do registro

A cada erro futuro registrar prompt disponível, resposta literal ou síntese, erro, causa, arquivos, solução, teste e resultado. Problemas previstos pelo Apêndice B do PDF são referências; não são ocorrências deste desenvolvimento.